

S : Single Responsibility Principle
O : Open/Closed Principle
L : Liskov Substitution Principle
I : Interface Segregation Principle
D : Dependency Inversion Principle

Single Responsibility Principle

-> 각 클래스는 하나의 책임만 갖고 있어야 한다는 의미이다.

```
class UserService{
public:
    void saveUser(const User& user){
        std::cout<<"User saved to database: "<<user.name<<std::endl;
    }
    void sendWelcomeEmail(const User& user){
        std::cout<<"Welcome email sent to: "<<user.name<<std::endl;
    }
    void logUserActivity(const User& user){
        std::cout<<"Logging activity for user: "<<user.name<<std::endl;
    }
};
```

잘못된 코드의 예시이다. 하나의 클래스에서 user의 database를 관리하고, email을 보내고, log를 기록하는 각각의 일을 같이 하는 것이다. 이렇게 된다면 UserService 클래스의 역할이 분명해 지지가 않고 확장성과 유연성에도 제약이 생기게 된다.

```

class UserRepository{
public:
    void saveUser(const User& user){
        std::cout<<"User saved to database: "<<user.name<<std::endl;
    }
};
class EmailService{
public:
    void sendWelcomeEmail(const User& user){
        std::cout<<"Welcome email sent to: "<<user.name<<std::endl;
    }
};
class UserActivityLogger{
public:
    void logUserActivity(const User& user){
        std::cout<<"Logging activity for user: "<<user.name<<std::endl;
    }
};

```

이는 SRP를 개선한 코드이다. 각 클래스 마다 역할을 세부적으로 지정해 주고 각클래스에서 원하는 멤버들을 더 추가하기 편하게 되었다.

이것으로 아까말한 확장성과 유연성을 코드에 부여할 수 있게 되었다.

Open-Closed Principle

->각 클래스가 확장에는 열려 있어야 하고 변경에는 닫혀 있어야 한다는 원칙이다.

```

class ReportGenerator{
public:
    void generateReport(const User& user){
        if (user.document=="PDF"){
            std::cout<<"Generating PDF document..."<<std::endl;
        }
        else if (user.document=="HTML"){
            std::cout<<"Generating HTML document..."<<std::endl;
        }
    }
};

```

잘못된 코드이다. 이 코드는 유저의 document타입을 판단해서 각각의 코드를 수행한다. 만약 조건문 안에 있는 코드가 기하급수적으로 많아 지게 된다면, 또는 다른 파일형식을

더욱 추가하게 된다면 문제가 생길 수가 있다. 아무래도 수정하면서 다른 코드들을 건드릴 위험과 충돌의 위험이 있다. 또한 매개변수들을 각 파일형식에 맞게 조정할 수가 없다 보니깐 거기에 따른 코드가 추가로 생겨날 수 있어서 함수 자체가 무거워질 수 있다. 이러한 문제점을 해결해 보자면

```
class Report{
public:
    void generate();
};
class PDFReport : public Report{
public:
    void generate(){
        std::cout<<"Generating PDF report..."<<std::endl;
    }
};
class HTMLReport : public Report{
public:
    void generate(){
        std::cout<<"Generating HTML report..."<<std::endl;
    }
};
class XMLReport : public Report{
public:
    void generate(){
        std::cout<<"Generating XML report..."<<std::endl;
    }
};
```

이렇게 해결해 볼 수 있다. 이 코드에는 자료의 형식을 판단하는 코드는 없지만 부모클래스의 Report를 상속받아서 각 파일형식의 클래스에서 각각 함수를 따로 적는 것이다. 이렇게 된다면 알아보기도 편하고 코드를 수정하는데 리스크를 감수할 필요성도 없어지게 된다.

Liskov Substitution Principle

->3번째는 LSP로 자식클래스는 언제나 부모 클래스를 대체할 수 있어야 한다는 원칙이다.

```

class Bird{
public:
    void fly(){
        std::cout<<"Bird is flying"<<std::endl;
    }
};
class Penguin : public Bird{
public:
    void fly(){
        std::cout<<"Penguins cannot fly"<<std::endl;
    }
};

```

잘못된 코드이다. 여기서 Penguin은 부모클래스인 Bird를 대체하지 못한다. 정확히는 최소한 자식클래스는 부모클래스가 행동하던 방식은 똑같이 해야한다는 것이다. 이 코드에서 Bird는 fly()에서 날 수 있다고 나왔지만 Penguin은 cannot fly, 날 수 없다고 나와있다. 이것이 단순히 출력이 아니라 모터의 설정이었다면 문제는 커질 수 있다.

```

class Flyable1{
public:
    virtual void fly1()=0;
};
class Bird1{
public:
    void eat1(){
        std::cout<<"Bird is eating"<<std::endl;
    }
};
class Sparrow1 : public Bird1 , public Flyable1{
public:
    void fly1(){
        std::cout<<"Sparrow is flying"<<std::endl;
    }
};
class Penguin1 : public Bird1{
};

```

그래서 여기서는 부모클래스를 독립해서 구분해주는 것이다. 대체할 수 있는 부모로만 한정하여서, Sparrow를 추가하였다. 이 새는 날수도 있고 먹을 수도 있는 조류이다. 그러므로 날수 있다는 flyable과 bird의 먹을 수 있다는 것을 동시에 상속받을 수 있다. 하지만 penguin은 먹을 수는 있지만 날 수 있는 없기 때문에 bird만 상속을 받았다.

java에서는 interface지만 c++에서는 순수가상함수로 interface를 제작하였다.

Interface Segregation Principle

->클래스는 자신이 사용하지 않을 메소드(멤버)를 구현하도록 강요 받지 말아야 한다는 원칙이다.

```
class Worker{
public:
    virtual void work()=0;
    virtual void eat()=0;
};

class Employee : public Worker{
public:
    void work(){
        std::cout<<"Employee is working"<<std::endl;
    }
    void eat(){
        std::cout<<"Employee is eating"<<std::endl;
    }
};

class Robot : public Worker{
public:
    void work(){
        std::cout<<"Robot is working"<<std::endl;
    }
    void eat(){
        std::cout<<"Robot do not eat"<<std::endl;
    }
};
```

잘못된 코드이다. 이 코드에서는 Robot클래스는 Worker클래스를 상속받아 하지도 않는 행동인 eat()을 구현해야만 한다.

```

class Workable{
public:
    virtual void work()=0;
};
class Eatable{
public:
    virtual void eat()=0;
};
class Employee1 : public Workable, public Eatable{
public:
    void work(){
        std::cout<<"Employee is working"<<std::endl;
    }
    void eat(){
        std::cout<<"Employee is eating"<<std::endl;
    }
};
class Robot1 : public Workable{
public:
    void work(){
        std::cout<<"Robot is working"<<std::endl;
    }
};

```

그래서 상속 클래스의 역할을 나눠서 해당하는 부분만 상속하는 방법을 사용하면 된다. 기존의 Worker클래스에서 Workable클래스와 Eatable클래스를 따로 나누어 Employee는 Workable, Eatable 둘다 해당되니깐 모두 상속받고, Robot 클래스는 Workable 클래스만을 상속 받으니까 eat()함수를 구현하지 않아도 된다.

<느낀점>

이 동영상은 단순히 SOLID원칙을 배우고 코드에 적용하는 것이 다가 아니라고 생각한다. 더 나아가서 이제 막 C++(객체지향언어)를 배운 사람들이 기본적으로 어떤 방향으로 클래스를 사용하면 좋은지에 대해 잘 알려주는 동영상이라고 느꼈다.

C를 지금까지 주언어로 사용하다 보니 클래스라는 개념에 대해서 잘 사용해 보지 못하여서 어렵다고 느꼈다. 이렇게 사용하는 것이 맞는지 조금 혼동을 느낄 때가 있었다. 하지만 이제는 한 클래스에 너무 많은 기능이 구현되어 있으면 나눠서 관리하고 상속에 대한 부분도 어떤 부분을 신경써야지 부모, 자식 클래스간의 충돌이 생기지 않는지 알게 되었다. 또한 필요없는 부분까지 상속을 받을 때는 어떻게 처리해야하는 지에 대해 동영상

상을 보고 직접 코드를 짜보았더니 기본적으로 클래스를 사용하면서 생겼던 문제들에 대해서 내가 스스로 생각해볼 수 있는 기본적인 견해가 생겼다.